Name : **Ebad Junaid**

Roll no : **BSCS 22046**

Course : **Information Security**

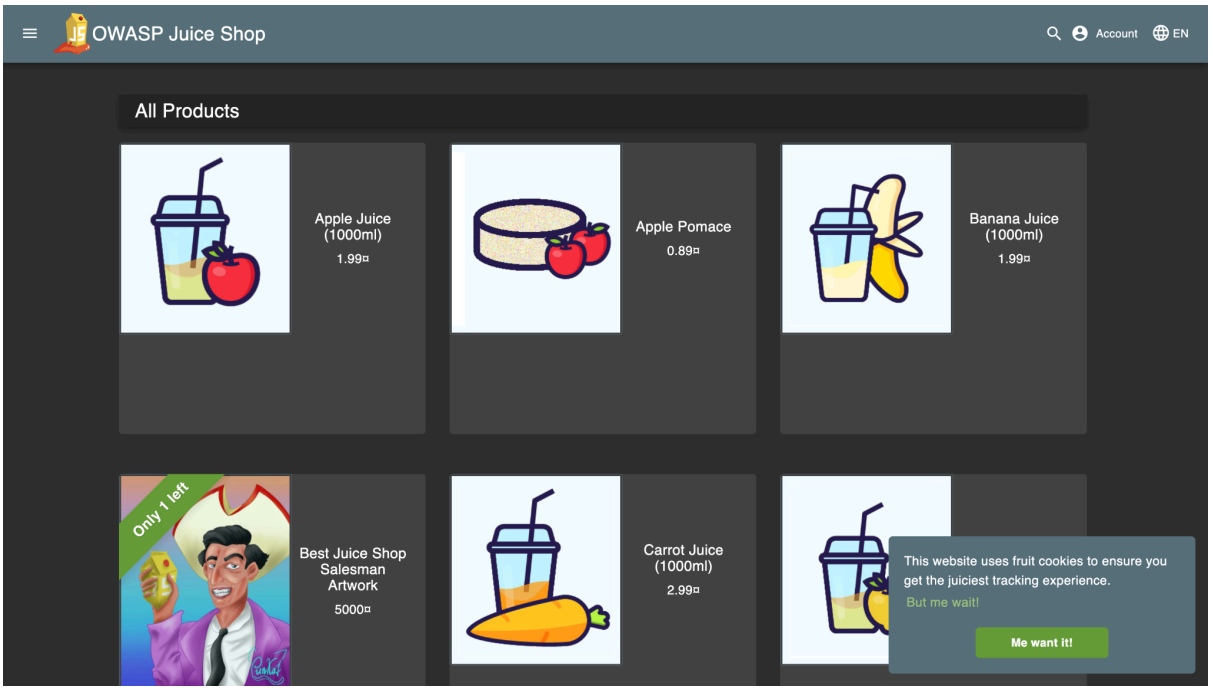
Submission Date : **March 13, 2026**

Part 1: Environment Setup

[EXPLANATION]: I selected the **OWASP Juice Shop** application for this web application penetration testing lab. I successfully deployed the application locally using Docker by pulling the official image and mapping it to port 3000 on my host machine.

[COMMAND]: `docker run --rm -p 3000:3000 bkimminich/juice-shop`

[SCREENSHOT]:



Caption: OWASP Juice Shop homepage successfully loaded via a local Docker container on localhost:3000.

[FINDINGS]: The application successfully initialises and the web interface is accessible via a local web browser.

- The homepage reveals a modern e-commerce storefront structure with a search bar and an "Account" section, providing immediate input vectors for future testing.
- Upon loading, the application displays a customized cookie tracking consent banner ("fruit cookies"), indicating the use of client-side storage and session tracking mechanisms that could be investigated further.

Part 2: OWASP Top 10 2025 Mapping & Reconnaissance

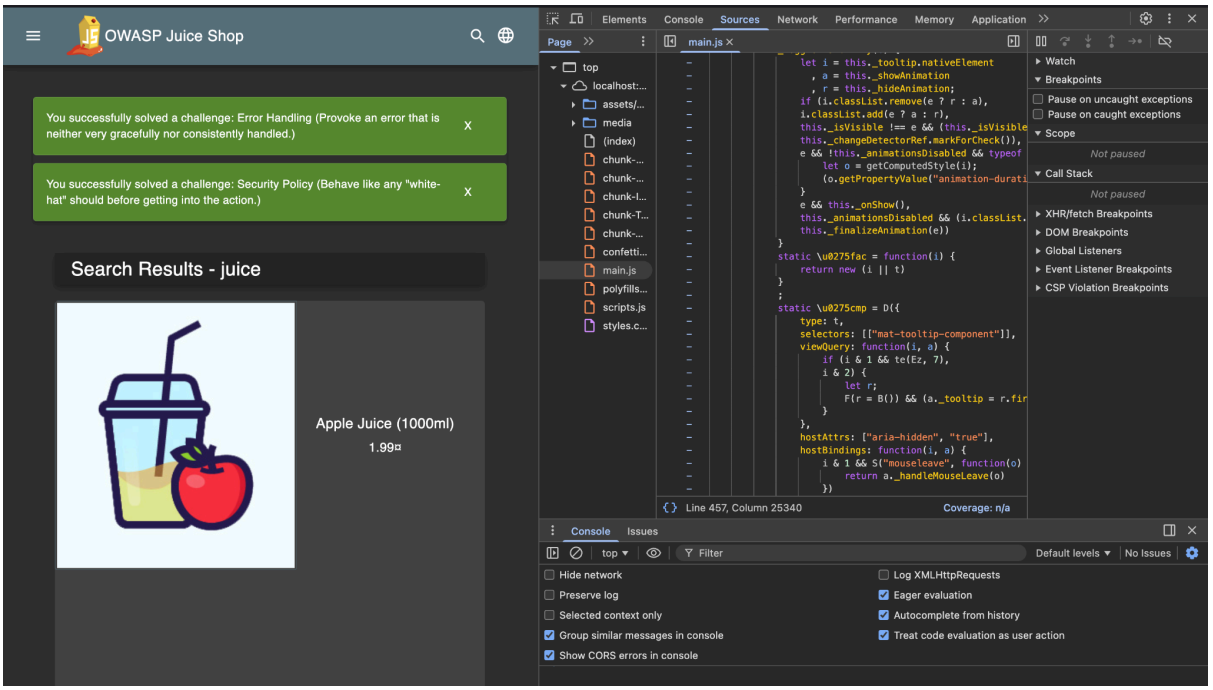
Vulnerabilities prediction :

- **A05:2025 Injection**: I expect to find injection vulnerabilities (such as SQLi or XSS) because the application features dynamic user inputs, including a search bar and login forms, which likely interact directly with a backend database. (Note: Referenced as A03 in the assignment document).
- **A07:2025 Authentication Failures:** Since the application functions as an e-commerce storefront allowing users to create accounts, log in, and manage profiles, there is a high probability of weaknesses in how user sessions and credentials are handled.
- **A02:2025 Security Misconfiguration:** Given the complexity of the application and its reliance on a modern client-side framework, it is likely that insecure default settings, verbose error messages, or unprotected APIs have been left exposed. (Note: Referenced as A05 in the assignment document).

[EXPLANATION]: I conducted reconnaissance using both manual inspection via browser Developer Tools and automated passive scanning using Nikto. This combined approach allowed me to map the application's architecture, identify potential hidden directories, and discover misconfigurations without actively attacking the system.

[COMMAND]: `nikto -h http://localhost:3000`

[SCREENSHOT]:



Caption: Manual inspection of client-side source code (main.js) using Browser Developer Tools.

```
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Your Nikto installation is out of date. Please run 'git pull' to update to the latest version of Nikto.
+ Target IP:          127.0.0.1
+ Target Hostname:    localhost
+ Target Port:        3000
+ Platform:           Unknown
+ Start Time:         2026-03-10 22:12:01 (GMT5)
---------------------------------------------------------------------------
+ Server: No banner retrieved
+ [999986] /: Retrieved access-control-allow-origin header: *.
+ [999100] /: Uncommon header(s) 'x-recruiting' found, with contents: /#/jobs.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999996] /robots.txt: contains 1 entry which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [001535] /.psql_history: This might be interesting.
+ [001675] /ftp/: This might be interesting.
+ [001811] /public/: This might be interesting.
+ [002739] /.htpasswd: Contains authorization information.
+ [002743] /.bash_history: A user's home directory may be set to the web root, the shell history was retrieved. This should not be accessible via the web.
+ [002748] /.mysql_history: Database SQL?.
+ [002756] /.sh_history: A user's home directory may be set to the web root, the shell history was retrieved. This should not be accessible via the web.
+ [007129] /.sqlite_history: This might be interesting.
+ [007203] /userdata.json: This might be interesting.
+ [007204] /login.json: This might be interesting.
+ [007205] /master.json: This might be interesting.
+ [007206] /masters.json: This might be interesting.
+ [007207] /connections.json: This might be interesting.
+ [007208] /connection.json: This might be interesting.
+ [007210] /PasswordsData.json: This might be interesting.
+ [007211] /users.json: This might be interesting.
+ [007212] /conndb.json: This might be interesting.
+ [007213] /conn.json: This might be interesting.
+ [007215] /accounts.json: This might be interesting.
+ [007303] /JAMonAdmin.jsp: JAMon - Java Application Monitor Admin interface identified. Versions 2.7 and earlier contain XSS vulnerabilities. See: https://cve.mitre.org/cgi-bin
/cvename.cgi?name=CVE-2013-6235
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https
://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8637 requests: 2 errors and 28 items reported on the remote host
+ End Time:           2026-03-10 22:14:10 (GMT5) (129 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Caption: Automated passive vulnerability scan using Nikto revealing missing security headers and exposed directories.

[FINDINGS]:

- **Manual Inspection - Tech Stack:** Inspection of the client-side source code (`main.js`) via Developer Tools confirmed the application uses the Angular framework for the frontend. Furthermore, the presence of # in the URL routing indicates a Single Page Application (SPA) architecture.
- **Manual Inspection - Sensitive Directory Discovery:** Navigating manually to `http://localhost:3000/robots.txt` revealed a `Disallow: /ftp` directive. Manually browsing to this directory confirmed it is accessible and contains potentially sensitive files.

- **Manual Inspection - Session Management:** The prompt for "fruit cookies" upon loading the site indicates the application relies on client-side storage mechanisms for tracking or session management.
- **Automated Passive Scan - Security Misconfiguration:** Nikto identified several missing security headers, including `Content-Security-Policy` and `X-Content-Type-Options`.
- **Automated Passive Scan - Information Leakage:** The scan flagged numerous potentially sensitive JSON configuration files and discovered an uncommon `x-recruiting` header, providing additional vectors for further investigation.

Link to Predicted OWASP Category:

The missing security headers discovered by Nikto directly link to our predicted **A02:2025 Security Misconfiguration** category *(Note: Referenced as A05 in the assignment document)*, as the server is failing to enforce standard client-side protections.

Part 3: Vulnerability Exploitation

A03:2025 Injection (SQL Injection)

[VULNERABLE FEATURE]: The user authentication mechanism on the login page *(/#/login)*, specifically within the "Email" input field.

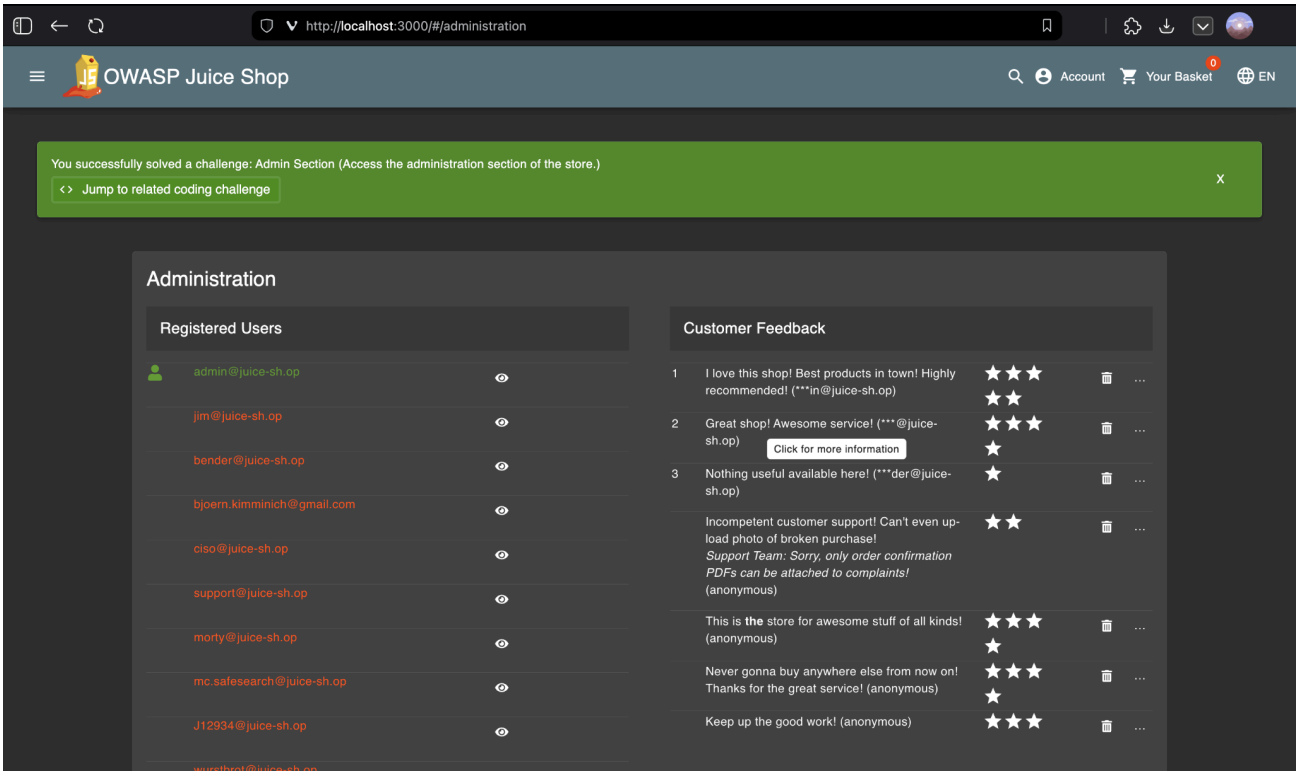
[EXPLANATION]: I tested the login form for improper input sanitization. First, I injected a single quote (') into the email field, which returned an unhandled `[object Object]` UI error, confirming the backend

database query syntax was disrupted. I then deployed a classic boolean tautology payload to force the SQL `WHERE` clause to evaluate to true, effectively bypassing password verification. Upon successful login, the UI did not display administrative links, so I manually navigated to the `/#/administration` endpoint to confirm privilege escalation.

[PAYLOAD]:

- **Unsuccessful Attempt: * Email: `'`**
  - **Password:** `test`
- **Successful Exploit: * Email: `' or 1=1--`**
  - **Password:** 12345

[SCREENSHOT]:



Caption: Successful authentication bypass via SQL Injection, confirming administrative privilege escalation by directly accessing the hidden Administration dashboard.

[IMPACT]: This exploit demonstrates a critical authentication bypass. By manipulating the SQL query, I was able to log in as the default administrator (`admin@juice-sh.op`) without a valid password. Navigating to the hidden administration panel confirmed full administrative privileges, granting unauthorized access to the internal user registry and customer feedback systems, representing a complete compromise of system confidentiality and integrity.

A03:2025 Injection (Cross-Site Scripting) / A07:2025 Identification and Authentication Failures

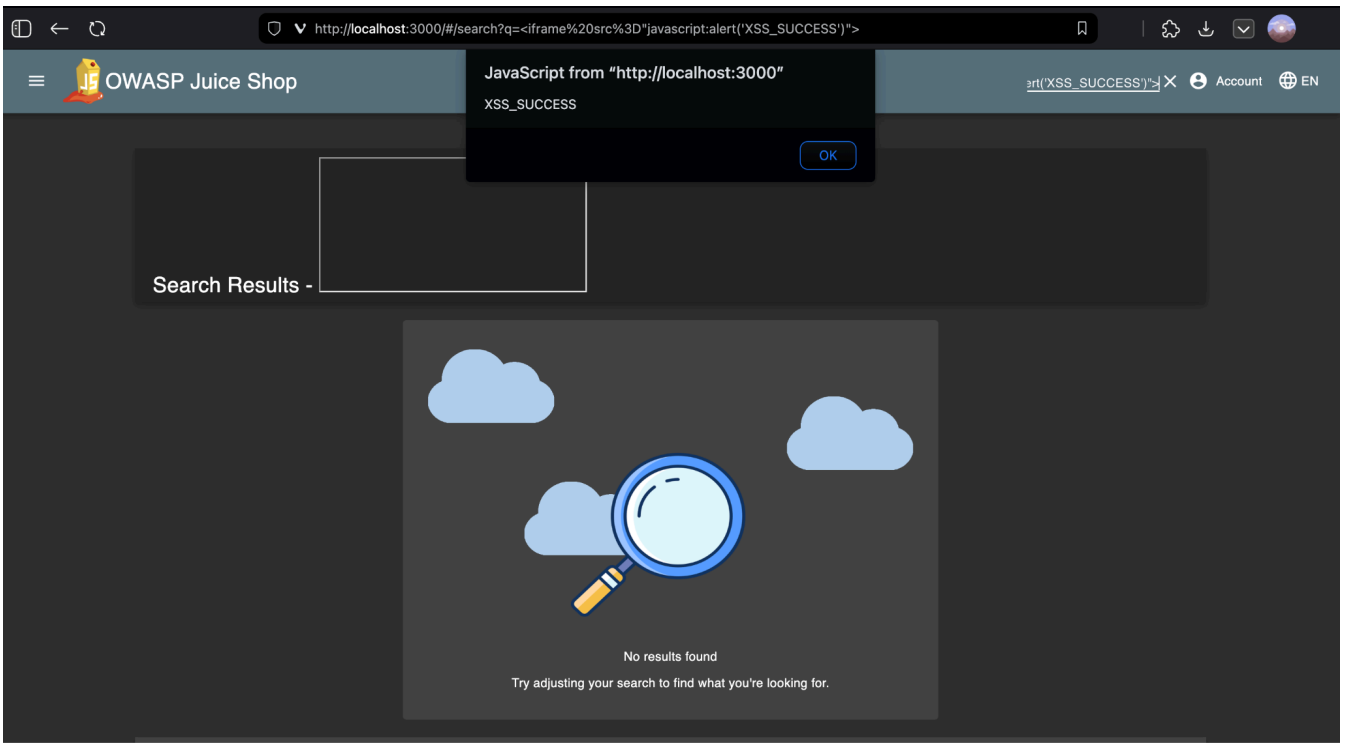
[VULNERABLE FEATURE]: The application's global product search functionality, specifically the search bar input field (which passes data to the `?q=` URL parameter).

[EXPLANATION]: I tested the search feature to determine if user-supplied input was properly sanitized before being reflected back onto the page. Initially, I attempted a standard `<script>` tag payload, which failed to execute, likely due to basic filtering or the frontend framework's default protections. To bypass this, I injected an HTML `<iframe>` tag containing a JavaScript URI. The application failed to properly encode this input, embedding the raw HTML payload directly into the Document Object Model (DOM), which forced the browser to execute the malicious script.

[PAYLOAD]:

- **Unsuccessful Attempt:** `<script>alert('XSS')</script>`
- **Successful Exploit:** `<iframe src="javascript:alert('XSS_SUCCESS')">`

[SCREENSHOT]:



Caption: Successful execution of a Reflected Cross-Site Scripting (XSS) payload via the product search bar, resulting in a JavaScript alert dialog.

[IMPACT]: This exploit demonstrates a critical Reflected XSS vulnerability. By crafting a malicious link containing this payload and tricking a victim into clicking it, an attacker can execute arbitrary JavaScript directly within the victim's browser session. In a real-world scenario, this can be weaponized to steal sensitive session tokens (like the previously discovered "fruit cookies"), perform actions on the user's behalf without consent, or redirect them to phishing sites.

Part 4: Mitigation Strategies

Mitigation for SQL Injection :

[VULNERABILITY]: A03:2025 Injection (SQL Injection).

[EXPLANATION]: The original code was vulnerable because it likely used insecure string concatenation or formatting to build SQL queries directly from untrusted user input (the email and password fields). This allowed the database interpreter to confuse the user's input (the `' or 1=1--` payload) as actual executable SQL commands, altering the query's underlying logic.

[FIX]: The most effective secure coding practice to prevent SQL Injection is the use of Parameterized Queries (also known as Prepared Statements). This technique forces the developer to define the SQL code first, and then pass the user inputs strictly as parameters later. The database driver ensures the input is treated exclusively as literal data, not as executable syntax.

[CODE EXAMPLE]: Python `sqlite3` implementation:

VULNERABLE APPROACH (String formatting)

`# The attacker payload breaks out of the quotes to alter the query.`
 	
`cursor.execute(f"SELECT * FROM users WHERE email='{email}' AND password='{pwd}'")`

SECURE APPROACH (Parameterized Query)
`# The ? placeholders ensure input is treated purely as data.` **`cursor.execute("SELECT * FROM users WHERE email=? AND password=?", (email, pwd))`**

[IMPACT]: Parameterization completely prevents the injection. If an attacker inputs `' or 1=1--`, the database will simply search for a user whose literal email address string is exactly `"' or 1=1--"`, which will safely return zero results and block the authentication bypass.

Mitigation for Cross-Site Scripting :

[VULNERABILITY]: A03:2025 Injection (Cross-Site Scripting).

[EXPLANATION]: The search functionality was vulnerable because it took untrusted input from the URL (`?q=`) and reflected it directly into the Document Object Model (DOM) without proper sanitization. By not neutralizing HTML characters, the browser interpreted the injected `<iframe>` tag as valid structural code rather than just text to be displayed.

[FIX]: The correct defensive control is Context-Aware Output Encoding. Before any user-supplied data is rendered in the browser, the backend template engine or frontend framework must convert HTML special characters into their safe HTML entity equivalents (e.g., converting < to `<`).

[CODE EXAMPLE]: Python implementation:

VULNERABLE APPROACH

```
# Using the ' | safe' filter explicitly tells the engine NOT to encode the data.

html_output = f"<h1>Search results for: {{ user_query | safe }}</h1>"
```

SECURE APPROACH

```
# Most modern Python templating engines (like Jinja2) auto-escape by default.
# If doing it manually in raw Python, use the html library:

import html
safe_query = html.escape(user_query)
html_output = f"<h1>Search results for: {safe_query}</h1>"
```

[IMPACT]: Output encoding strips the input of its executable context. When the encoded data is sent to the browser, the payload `<iframe src...>` is rendered visibly on the screen as plain text, rather than being parsed and executed as a malicious script, thereby neutralizing the XSS vulnerability.

Broader OWASP Controls

[VULNERABILITY]: A02:2025 Security Misconfiguration *(Note: Referenced as A05:2025 in the assignment document)*.

[EXPLANATION]: During the passive reconnaissance phase in Part 2, the automated Nikto scan revealed that the web server is missing critical HTTP security headers, leaving the application unnecessarily exposed to client-side vulnerabilities.

[RECOMMENDATION & JUSTIFICATION]: I recommend implementing robust security headers at the server level, specifically a strict `Content-Security-Policy` (CSP) , `X-Content-Type-Options`. I am recommending this mitigation because a strong CSP acts as a critical layer of defense-in-depth. Even if a developer accidentally introduces an input validation flaw elsewhere in the application, the `Content-Security-Policy` will block the browser from executing unauthorized inline scripts or loading malicious external resources, significantly reducing the overall risk surface of the application.

Part 5: Analysis & Reflection

[REFLECTION]: This lab was an eye-opening journey into web application security. The OWASP Top 10 2025 framework acted as a direct roadmap for my assessment. By understanding the top risk categories beforehand, I knew exactly what to predict, which guided my manual reconnaissance and helped me make sense of my automated

Nikto scan results. The potential real-world impact of the vulnerabilities I actively exploited is severe. With a basic SQL injection payload (`' or 1=1--`), I completely bypassed authentication and accessed the hidden administration dashboard which proves that an attacker could easily cause a massive data breach or take over the entire system. Furthermore, bypassing the search bar's filter with an XSS `<iframe>` payload demonstrated how easily attackers can hijack user sessions. My biggest takeaway regarding the relationship between development practices and security is that developers cannot rely entirely on front-end frameworks for protection. Even though the application blocked my standard `<script>` tag, the underlying flaw remained. True security requires intentional backend practices like using parameterized queries and strict output encoding to fix the root cause rather than just patching the symptoms.